CHAPTER 1 • GIT FUNDAMENTALS ~15 Min

Student

Can you explain what this chapter is about?

Claude • AI Assistant

Welcome to Week 7. Across the previous six weeks you have built real projects - Python programs, machine-learning notebooks, deep-learning models, Snowflake pipelines, RAG systems.

You have been pushing to GitHub the entire time, but most students do this the wrong way: dragging files into the web UI, no branches, no commit messages worth reading, no automated tests. **This week is where all of that becomes professional**.

By Friday, your projects will look and behave the way engineering codebases actually do.

Chapter 1 (Git Fundamentals) starts from zero - assuming no real version-control knowledge. **Section 1.1** makes the case for version control by naming the four capabilities every professional codebase depends on.

Section 1.2 cleans up the near-universal beginner confusion between Git (the local tool) and GitHub (the hosted platform), and gets you set up with SSH auth. **Section 1.3** teaches the three areas Git tracks - working directory, staging, commit history - and the day-to-day loop of `git add` -> `git commit` -> `git push` .

Section 1.4 covers repositories and remotes: clone, push, pull, fetch, multiple remotes, and the fork-based flow used to contribute to open-source. **Section 1.5** locks in the two habits that separate hobbyist repos from professional ones: `.gitignore` (keep secrets and bloat out) and commit-message hygiene (the 50/72 rule and imperative mood).

Section 1.6 consolidates with a self-check and the AskMyNotes baseline exercise.

Student

Can you break down the next concept?

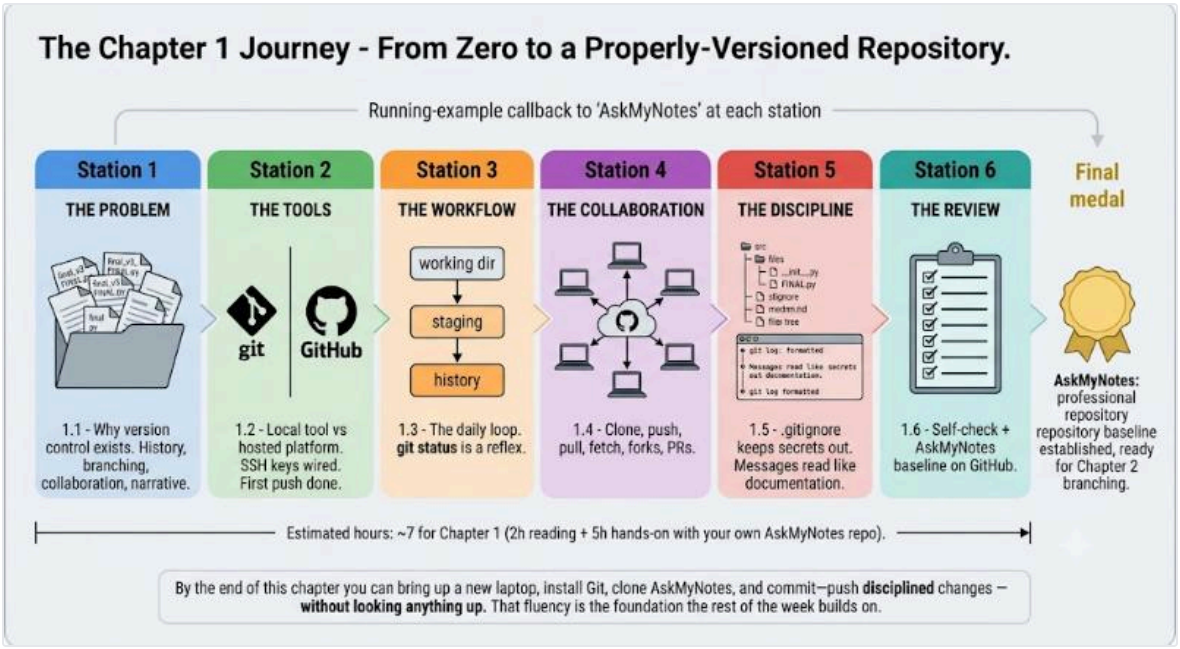
Claude • AI Assistant

Running example: AskMyNotes. Across all five chapters this week, we use AskMyNotes - the RAG-based PDF Q&A app from earlier sessions - as the concrete vehicle.

It has application code (FastAPI backend, small front-end), ML components (chunking, embedding, retrieval, LLM call), tests worth writing, a deployment story, and a model lifecycle. Every abstract concept in the week is tied to a command you actually run on the AskMyNotes repo.

By the end of Chapter 1, you should be able to bring up a fresh laptop, install and configure Git, clone AskMyNotes from GitHub, make disciplined commits with high-quality messages, and push them back - all without looking anything up. That is the baseline.

Chapter 2 turns it into professional collaboration: branching strategies, rebasing vs merging, resolving conflicts (the most anxiety-inducing Git skill and worth extra practice), Pull Request etiquette, and Git internals so nothing feels like magic and nothing is unrecoverable.



The Chapter 1 Journey - From Zero to a Properly-Versioned Repository

Chapter 1 at a Glance

Subsection	Title	Core Question Answered	You Can Now...
1.1	What is Version Control?	Why do professionals use Git at all?	Explain the four superpowers - history, branching, collaboration, narrative - to any beginner
1.2	Git vs GitHub	What's the difference between the two?	Set up Git locally, configure identity, wire an SSH key, and push your first repo to GitHub

Subsection	Title	Core Question Answered	You Can Now...
1.3	The Git Workflow	How do the three areas fit together?	Run the edit -> add -> commit -> push loop fluently, and undo mistakes at each level
1.4	Repositories and Remotes	How do commits move between machines?	Clone, push, fetch, pull, add multiple remotes, and follow the fork-based OSS contribution flow
1.5	.gitignore and Commit Hygiene	How do I keep the repo clean and readable?	Write a comprehensive .gitignore, follow the 50/72 commit-message rule, and recover from a leaked secret
1.6	Chapter Review + Summary	What do I own now?	Baseline the AskMyNotes repo on GitHub with proper hygiene, ready for Chapter 2's professional branching

bash — Chapter 1 preview - the 25 commands you'll type every day by Friday

```
# --- Setup (once per machine) ---
git config --global user.name  "Priya Sharma"
git config --global user.email "priya.sharma@kmit.in"
ssh-keygen -t ed25519 -C "priya.sharma@kmit.in"      # add pub key to GitHub

# --- Start / join a project ---
git init                                          # new local repo
git clone git@github.com:me/askmynotes.git      # copy existing repo

# --- The daily loop ---
git status
git diff
git add chunker.py tests/test_chunker.py
git diff --staged
git commit -m "Snap chunk boundaries to whitespace to avoid splitting words"
git log --oneline -10
git push

# --- Stay in sync (start and end of each session) ---
git pull --rebase
```

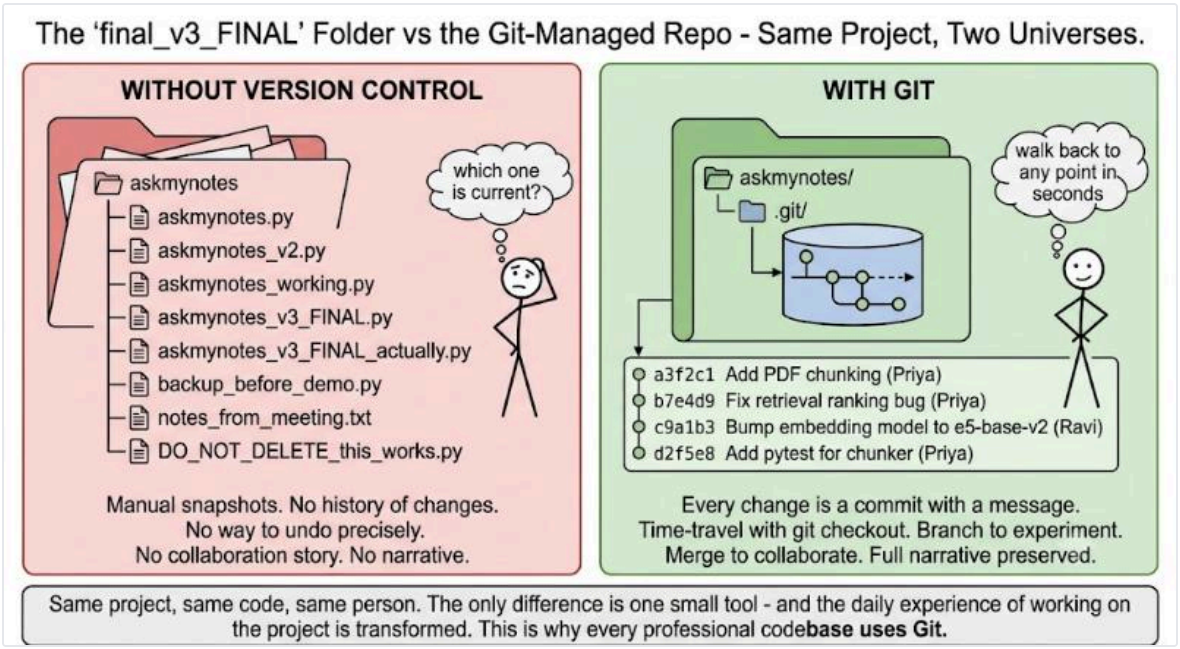
```
# --- Inspect, undo, recover ---
git restore <file>                              # discard working-dir edits
git restore --staged <file>                     # unstage a file
git commit --amend -m "Better subject"          # fix LAST commit (unpushed!)
git revert <commit>                             # safe undo of pushed commit

# --- Remotes ---
git remote -v
git remote add origin git@github.com:me/askmynotes.git
git push -u origin main                         # first push sets upstream

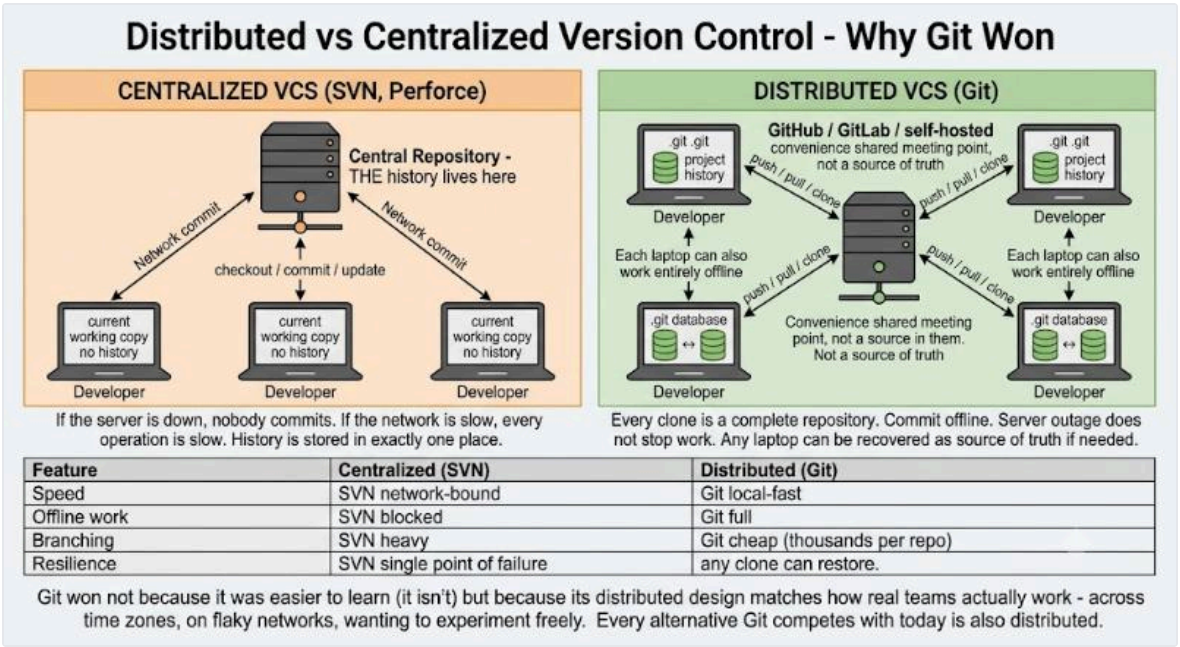
# --- Hygiene (details in 1.5) ---
cat > .gitignore <<'EOF'
__pycache__/
.venv/
.env
.pytest_cache/
data/
models/*.bin
.DS_Store
EOF
git add .gitignore
git commit -m "Add .gitignore for Python + ML project"
git push
```

WHAT IS VERSION CONTROL?

Claude · AI Assistant



The 'final*v3*FINAL' Folder vs the Git-Managed Repo - Same Project, Two Universes



Distributed vs Centralized Version Control - Why Git Won

bash — Preview - what a real Git workflow looks like (we build up to this all week)

```
# You're about to add a new feature to AskMyNotes - a re-ranker step in
retrieval.
# Instead of editing main directly, you create a branch:
git checkout -b feature/rerank-retrieval

# You edit files (retrieval.py, tests/test_rerank.py), then check what
changed:
git status
```

```
git diff

# You stage and commit the changes with a meaningful message:
git add retrieval.py tests/test_rerank.py
git commit -m "Add cross-encoder re-ranker on top-K retrieval"

# You push the branch to GitHub:
git push -u origin feature/rerank-retrieval

# You open a Pull Request; teammates review; CI runs tests automatically.
# After approval, the branch is merged into main.

# Later, a bug is reported. You inspect what changed recently:
git log --oneline -10
# a3f2c1 Add cross-encoder re-ranker on top-K retrieval
# b7e4d9 Fix PDF chunk overlap off-by-one
# c9a1b3 Bump embedding model to e5-base-v2

# You narrow down the guilty commit and revert it:
git revert a3f2c1
git push

# Production is fixed in under two minutes. Nothing was manually copied.
# This is the day-to-day of a professional codebase. Every command
# above is unpacked over the next four subsections.
```

Student

Sir, over the past few weeks we've built a bunch of projects - the AskMyNotes RAG app, the deep-learning notebooks, the Snowflake ETL pipelines. I've been pushing everything to GitHub because you told us to, but honestly I just click 'Upload files' in the browser and drag things in.

Is that really what professionals do?

Claude • AI Assistant

Not remotely. You've been storing files on GitHub, which is a bit like using a hospital only for its parking lot.

GitHub is built on top of a system called **Git**, and Git is what makes a codebase professional. This week we're going to fix that gap - starting today with the concept, then

working up to disciplined branching, testing, continuous integration, and finally ML-model versioning by Friday.

By week's end your AskMyNotes repo will look and behave like something a hiring manager would take seriously.

Let's begin with the foundational question: **what problem does version control actually solve?**

THE 'FINAL*V3*FINAL.PY' PROBLEM

Student

Alright - what problem?

Claude • AI Assistant

CONCEPT CARD

Open any student's Downloads folder and you will find variations of this pattern:

code — Code Example

```
askmynotes.py
askmynotes_v2.py
askmynotes_working.py
askmynotes_v3_FINAL.py
askmynotes_v3_FINAL_actually.py
askmynotes_v3_FINAL_actually_use_this_one.py
askmynotes_backup_before_prof_review.py
```

Every filename is a manual snapshot. This 'system' has predictable failures:

- You lose track of which file is current.
- You cannot see **what changed** between v2 and v3 without diffing them by eye.
- You cannot go back to an intermediate state without hoping you kept the right file.
- You cannot collaborate - if two people edit v3, one person's work gets overwritten.
- You have no explanation of **why** any change was made.

Version control is the software solution to all five problems at once. It stores the entire history of a project inside a hidden folder (`.git`), snapshots your changes on demand (called **commits**), and lets you walk backwards, branch sideways, or replay any point in the history at will.

THE FOUR CAPABILITIES YOU GET FOR FREE

Student

Break it down - what exactly do I gain?

Claude • AI Assistant

CONCEPT CARD

Four superpowers that turn ad-hoc coding into engineering:

1. Time travel through your own work. Every commit is a permanent snapshot.

`git log` shows the timeline; `git checkout <commit>` walks you back to any past state. That embarrassing three-day detour into a bad architecture?

Gone in one command - but *preserved in history* so you can study why it didn't work.

2. Fearless experimentation via branches. Want to try a wild rewrite of AskMyNotes' embedding pipeline?

Create a **branch** - a parallel line of development. If it works, merge it back.

If it doesn't, delete the branch. Your main code was never touched.

This changes how you code: you stop being scared to try things.

3. Multi-person collaboration without stepping on toes. Two developers can work on the same repository simultaneously, on the same file even.

Git tracks their changes independently and merges them together, only asking a human to intervene when the same lines were edited on both sides (called a **conflict** - we'll master these in Chapter 2).

4. A narrative of the project. Every commit carries a **message** explaining why the change was made. Six months from now, `git log` reads like the diary of your project. Future-you (or a new teammate) can understand not just what the code does, but why it evolved this way.

CENTRALIZED VS DISTRIBUTED

Student

Are all version-control systems the same?

Claude • AI Assistant

CONCEPT CARD

Historically no. There were two families:

Centralized VCS (SVN, Perforce, CVS) - one central server holds THE history. Everyone checks out a working copy from the server; every commit goes back to the server. If the server is down, nobody works. If the network is slow, everything is slow.

Distributed VCS (Git, Mercurial) - **every developer has the full history** on their local machine. You commit locally without a network. You share by pushing to and pulling from remotes. If GitHub is down, you keep working; your local Git is complete.

Git won because 'distributed' means fast, offline-capable, and resilient. Modern software development is almost entirely Git today; the alternatives survive only in specialized shops.

WHERE GIT CAME FROM

Student

Any short history?

Claude • AI Assistant

Git was created by **Linus Torvalds** in 2005 to manage the Linux kernel after the kernel team lost access to their previous tool (BitKeeper). Linus wrote the first version in about two weeks.

The design goals were speed, integrity (every commit is content-addressed by a SHA-1 hash so tampering is detectable), and rock-solid support for **non-linear development** - meaning branches that can be merged, rebased, and rearranged fluidly. Twenty years later, Git is the near-universal standard - powering GitHub, GitLab, Bitbucket, and countless private servers - and every professional engineer is expected to be fluent in it.

THE RUNNING EXAMPLE - ASKMYNOTES

Student

What are we going to use as our example project this week?

Claude • AI Assistant

EXAMPLE CARD

AskMyNotes - the RAG-based PDF Q&A app you've been building in earlier sessions. It's an ideal week-long example because it has:

- **Application code** - a small FastAPI backend, a simple HTML/JS front end.
- **ML components** - a text embedder, a vector store, a retrieval loop, an LLM call.
- **Tests worth writing** - the chunking function, the retrieval ranking, the API response shapes.
- **A deployment story** - eventually it goes on a small server or a container platform.
- **A model lifecycle** - you may retrain or swap the embedding model, and you want to track which version served which query.

Every subsection this week, we'll do the concept then apply it to AskMyNotes. By Friday, the repo will have proper branches, tests, a CI pipeline that runs on every push, deployment on tagged releases, and MLflow tracking the embedding-model versions.

ANALOGY

Version control is like **track changes in Microsoft Word, but for entire folders of files, with the ability to fork the document into multiple parallel versions and merge them later**. Or, if you play video games with save states - it's save-states for your entire codebase, plus the ability to name each save with a message and share saves with teammates.

REAL WORLD APPLICATION

A junior engineer at an insurance startup shipped a fix to production on a Friday afternoon. Monday morning, customer reports flooded in - his fix had broken a downstream report.

With version control, the team ran `git log` on the affected file, found the exact commit that introduced the regression, and used `git revert` to undo it in about ninety seconds. Without version control, that would have been a Monday of frantic archaeology through backup zip files.

This is not an exceptional story - it happens somewhere every day. The invisible skill of version control is what turns 'production incident' into 'small blip'.

WARNING

Do not confuse **backing up files** with **version control**. Dropbox, Google Drive, and iCloud synchronize your latest state but discard granular history (or keep only limited version windows).

Git preserves every commit forever with intentional, human-authored messages. The two solve different problems - and for code, only Git works properly.

COMMON PITFALL / MISTAKE

Beginners think Git 'is a folder on GitHub.' It is not. Git is a **local tool** on your laptop that maintains history inside a hidden `.git` directory.

GitHub is where you push copies of that history to share and collaborate. You can use Git without GitHub (many teams do, with private servers or GitLab); you cannot use GitHub without Git.

We tease this apart carefully in 1.2.

EXAM TIP CARD

Interviewers love asking 'why do we use version control?' as a warmup. A crisp answer names all four superpowers: **history**, **branching**, **collaboration**, and **narrative**.

Then mention that Git is distributed (every clone has full history) as opposed to centralized systems like SVN, and you sound properly grounded.

PRACTICE QUIZ

You're working alone on AskMyNotes. Why should you still use Git even though nobody is collaborating with you?

A) Version control is only useful for teams

B) History, safe experimentation via branches, and a portfolio-visible narrative on GitHub

C) It automatically backs up your files to the cloud

D) It makes your Python code run faster

▼ Reveal Correct Answer

Answer: B. Even solo, you gain time-travel, fearless experimentation, and a public record of the project that recruiters can see. Backing up (C) is a side effect of pushing to GitHub, not the point. D is nonsense.

EXERCISE

On your laptop right now: (a) open your AskMyNotes folder, (b) list every file whose name contains 'final', 'v2', 'backup', or a date, (c) delete them all after screenshotting the list. In tomorrow's session we install Git and this class of chaos ends permanently.

AI DEEP DIVE PROMPT

> Ask an AI: '*Explain to a first-time programmer, using an analogy from everyday life, why version control is worth learning even for solo projects. Then list five things that go wrong without it.*'

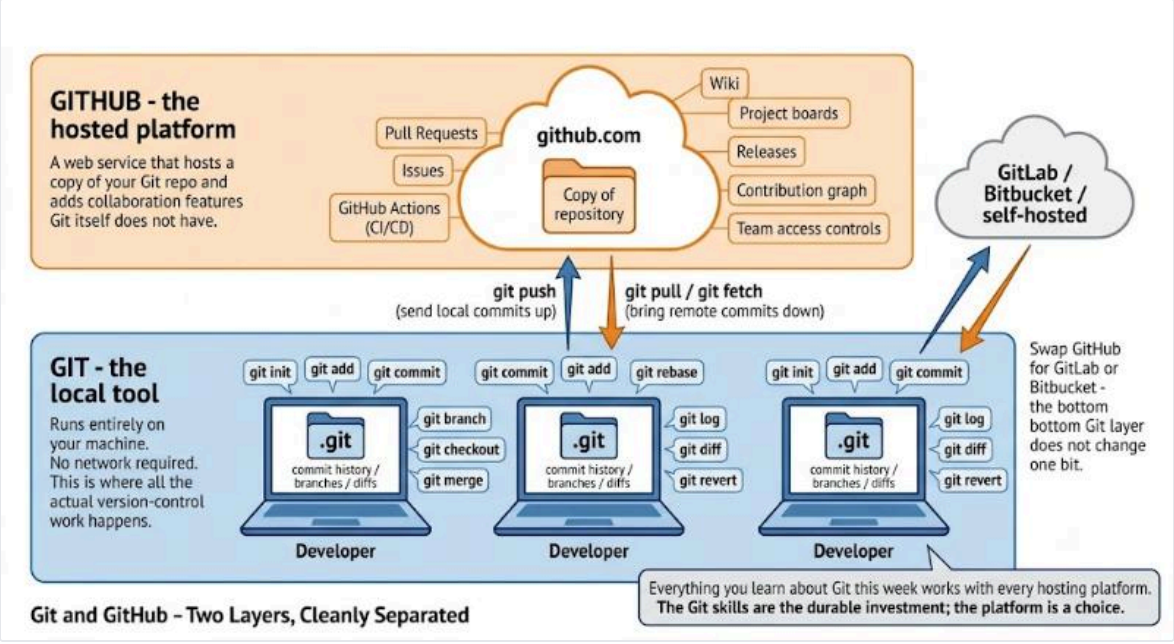
SUMMARY CARD

Version control solves the 'final*v3*FINAL.py' chaos by storing your project's full history in a `.git` folder, letting you: (1) walk backwards through commits, (2) branch to experiment safely, (3) collaborate without overwriting, (4) narrate your project's evolution with commit messages. **Git** is the distributed version-control system that beat centralized rivals because every clone is a complete repository - fast, offline-capable, resilient.

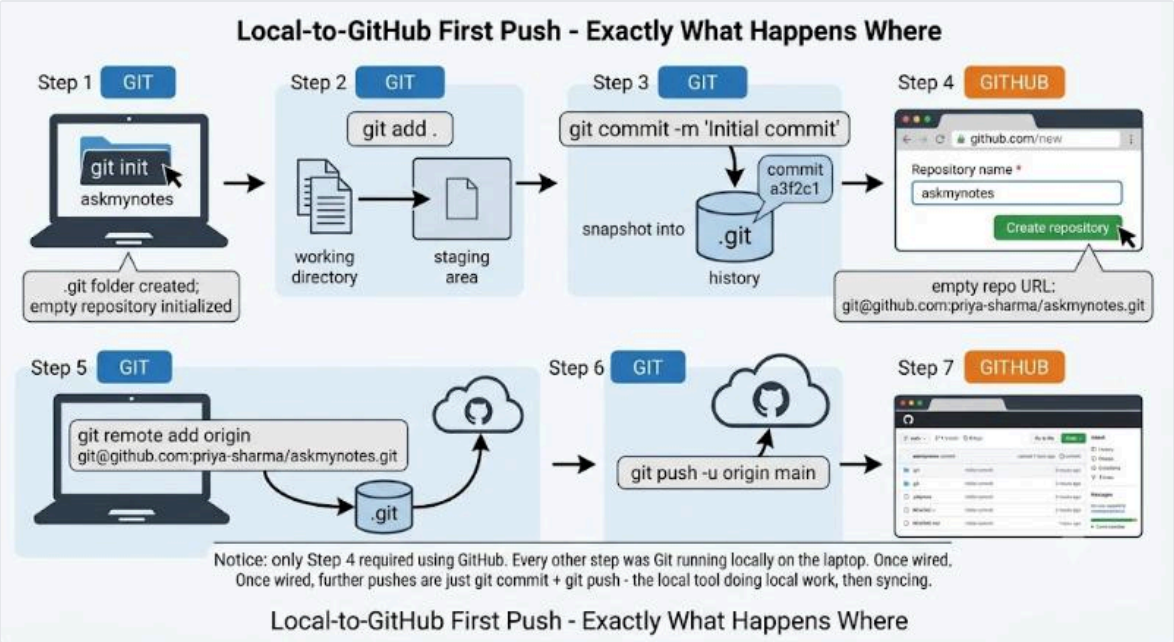
This week we'll take AskMyNotes from ad-hoc folder to properly-versioned, tested, CI-backed project.

GIT VS GITHUB - LOCAL TOOL VS HOSTED PLATFORM

Claude • AI Assistant



Git and GitHub - Two Layers, Cleanly Separated



Local-to-GitHub First Push - Exactly What Happens Where

bash — First-time Git + GitHub setup - the one-time ritual

```
# 1. Install Git
#    macOS:    brew install git
#    Windows:  download from git-scm.com/download/win
#    Linux:    sudo apt install git
git --version                              # verify

# 2. Configure identity (use your GitHub email!)
git config --global user.name  "Priya Sharma"
git config --global user.email "priya.sharma@kmit.in"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"     # VS Code editor
git config --global pull.rebase false               # merge on pull (safer
default)

# 3. Generate an SSH key for GitHub
ssh-keygen -t ed25519 -C "priya.sharma@kmit.in"
#   Accept default file path; optional passphrase

# 4. Copy the PUBLIC key (never share the private key!)
#    macOS:            pbcopy < ~/.ssh/id_ed25519.pub
#    Windows Git Bash: cat ~/.ssh/id_ed25519.pub | clip
#    Linux:            xclip -sel clip < ~/.ssh/id_ed25519.pub

# 5. Paste into: github.com -> Settings -> SSH and GPG keys -> New SSH key

# 6. Test the connection
ssh -T git@github.com
```

```
# Expected: Hi priya-sharma! You've successfully authenticated,
#           but GitHub does not provide shell access.

# 7. First push - wire an existing folder to a new GitHub repo
cd ~/projects/askmynotes
git init
git add .
git commit -m "Initial commit: baseline AskMyNotes app"
git branch -M main
git remote add origin git@github.com:priya-sharma/askmynotes.git
git push -u origin main

# 8. Verify on github.com/priya-sharma/askmynotes - your commit is live.
```

Student

You keep saying 'Git is a local tool and GitHub is a platform'. Can we really nail down the difference before I get any deeper?

Claude • AI Assistant

Yes - because half of every beginner's confusion about the whole topic dissolves once this distinction clicks. **Git** is a program on your laptop.

It creates history, makes branches, merges, reverts - all inside a `.git` folder that sits right next to your source code. **GitHub** is a website (owned by Microsoft) where you can host a *copy* of your Git repository so others can see it, collaborate on it, and where extra features like Pull Requests, issue tracking, and CI/CD pipelines live.

You use Git even if GitHub goes offline. You cannot use GitHub without Git.

THE TWO ROLES, PRECISELY

Student

Give me the crisp separation.

Claude • AI Assistant

CONCEPT CARD

Think of it as **tool vs venue**:

Git (the tool) does the version-control work. Commit, branch, merge, log, diff, revert, checkout, rebase - every command that changes or inspects history is a Git command.

Git runs entirely on your laptop; it needs no network. The `.git` folder inside your project holds everything Git needs.

GitHub (the venue) is where you publish your repository so it lives somewhere besides your laptop. On top of hosting, GitHub adds features Git itself does not have:

- **Pull Requests** - a web UI for proposing and reviewing changes.
- **Issues** - lightweight ticketing.
- **GitHub Actions** - automated pipelines (this week's Chapter 3).
- **Wiki, Projects, Releases, Security scanning** - project-management add-ons.
- **Social features** - stars, forks, followers, contribution graph.

None of these are Git. They are GitHub's value-add. If you switch to GitLab or Bitbucket tomorrow, your Git commands don't change; only the venue changes.

GITHUB'S COMPETITORS

Student

Are there alternatives to GitHub?

Claude • AI Assistant

CONCEPT CARD

Yes - and knowing the landscape helps in interviews:

- **GitLab** - very similar feature set, popular in enterprises, offers a strong self-hosted option (many companies run their own GitLab server for privacy).
- **Bitbucket** - Atlassian's offering, integrates tightly with Jira and Confluence.
- **Azure DevOps (formerly VSTS)** - Microsoft's enterprise offering, common in .NET shops.
- **Gitea, Codeberg** - lightweight open-source self-hosted options.

All host Git repositories. All offer Pull Requests, issues, CI/CD, and code review. The choice between them is usually driven by company preference or existing tooling - the underlying Git skills carry over perfectly.

INSTALLING GIT

Student

How do I actually get Git on my machine?

Claude • AI Assistant

EXAMPLE CARD

Depends on your OS:

macOS: `brew install git` (installs Homebrew first if you don't have it: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`). Alternatively, the Xcode Command Line Tools bundle includes Git.

Windows: download the installer from `git-scm.com/download/win` . During install, accept the default 'Git Bash' terminal - it gives you a Unix-like shell where all the Git tutorials on the internet will work verbatim.

Linux (Ubuntu / Debian): `sudo apt update && sudo apt install git` . On Fedora / RHEL: `sudo dnf install git` .

Verify with `git --version` - you should see something like `git version 2.42.0` .

ONE-TIME SETUP - IDENTITY

Student

What do I do after installing?

Claude • AI Assistant

EXAMPLE CARD

Tell Git who you are. This name and email are baked into every commit you make:

bash — Code Example

```
git config --global user.name  "Priya Sharma"
git config --global user.email "priya.sharma@kmit.in"
```

Use the **same email you use for your GitHub account** - GitHub uses that email to associate commits with your profile (so your contribution graph fills up). Also set a sensible default branch name and editor:

```bash
git config --global init.defaultBranch main
git config --global core.editor "code --wait"   # VS Code as editor
```

CREATING A GITHUB ACCOUNT

Student

What do I do on GitHub side?

Claude • AI Assistant

CONCEPT CARD

Three steps:

1. Sign up at `github.com` - free tier is more than enough for a student. Use a **professional-looking username** (your name, or a variant) - this URL will be on your resume. `luffy-strawhat-99` looks bad; `priya-sharma` looks fine.

2. Set up SSH authentication (recommended over password/token for daily use):

```bash
# Generate an SSH key
ssh-keygen -t ed25519 -C "priya.sharma@kmit.in"
# Press Enter to accept defaults; set a passphrase if you want extra
security

# Copy the public key to clipboard (macOS)
pbcopy < ~/.ssh/id_ed25519.pub
# Windows Git Bash:  cat ~/.ssh/id_ed25519.pub  |  clip
# Linux:             xclip -sel clip < ~/.ssh/id_ed25519.pub
```

Then paste it into GitHub -> Settings -> SSH and GPG keys -> New SSH key. Test the connection:

```bash
```

```
ssh -T git@github.com
# You should see: Hi priya-sharma! You've successfully authenticated...
```

3. Enable two-factor authentication on your GitHub account. This is not optional at a professional level - a compromised GitHub account is a serious incident.

THE 'FIRST PUSH' FLOW

Student

Show me the connection between the two.

Claude • AI Assistant

EXAMPLE CARD

A brand-new AskMyNotes repo, from local zero to on GitHub:

bash — Code Example

```
# Local side (Git)
cd ~/projects/askmynotes
git init                            # creates .git folder here
git add .                           # stage all files
git commit -m "Initial commit"      # first snapshot in history

# GitHub side (browser)
# 1. Create a new repository at github.com/new
# 2. Do NOT initialize with README/gitignore/license (you have local content
already)
# 3. Copy the SSH URL shown, e.g. git@github.com:priya-sharma/askmynotes.git

# Wire them together
git remote add origin git@github.com:priya-sharma/askmynotes.git
git branch -M main                  # rename current branch to main if
needed
git push -u origin main             # push local main to remote main, set
upstream
```

Now the same history exists in two places: your laptop and GitHub. Later, every `git push` pushes new commits to GitHub; every `git pull` fetches new commits from GitHub. The two stay in sync as long as you do this regularly.

ANALOGY

Git is like **the word processor on your laptop**; GitHub is like **the shared drive where you and your team keep the documents**. You do all the editing in the word processor; the shared drive is where you publish and collaborate.

Nobody would confuse Microsoft Word with SharePoint - but beginners often confuse Git with GitHub because the two are almost always used together.

REAL WORLD APPLICATION

A group of five students building a hackathon project ran into a subtle problem: GitHub had a two-hour outage right before the demo. Two teams panicked because they had never committed locally - they'd been dragging files into the GitHub web UI.

The one team using proper `git commit` + `git push` kept working locally the whole outage, then pushed everything once GitHub came back. Their project shipped; the others didn't.

Distributed VCS earned its keep in one evening.

WARNING

Never store passwords, API keys, `.env` files, or any secret credentials in a Git repo - especially a **public** one. GitHub scans public repos for exposed secrets and will notify the owner (and, for AWS keys, notify AWS who may auto-revoke), but by then bots have already scraped your key.

Section 1.5 covers `.gitignore` to prevent this. If you have leaked a key: **rotate it immediately**, don't just delete the file - Git history preserves it forever.

COMMON PITFALL / MISTAKE

Using the GitHub web UI 'Upload files' button as your primary way to add code. This works but it's crippling: you get no local history, cannot branch, cannot use CI properly, cannot work offline, and your commits all say 'Add files via upload' - useless as history.

Learn `git add` / `git commit` / `git push` from day one; treat the web upload as an emergency-only escape hatch.

EXAM TIP CARD

Interview classic: '*What's the difference between Git and GitHub?*' Perfect answer: 'Git is a distributed version-control system - a local tool that manages history in a `.git` folder. GitHub is a hosted platform for sharing Git repositories, plus a layer of collaboration features - Pull Requests, issues, Actions - built around Git.

You use Git locally; you use GitHub to publish and collaborate. Alternatives to GitHub include GitLab and Bitbucket.'

PRACTICE QUIZ

Which of these is a Git operation, not a GitHub feature?

A) Opening a Pull Request

B) Running GitHub Actions CI on every push

C) `git commit -m "fix bug"`

D) Filing an issue with reproduction steps

▼ Reveal Correct Answer

Answer: C. Commit is pure Git - it works locally with no network. A, B, D are all GitHub features layered on top of the Git repository.

EXERCISE

On your laptop: (1) install Git, (2) configure your name/email with the same email you use on GitHub, (3) generate an SSH key and add it to GitHub, (4) create a new empty repository at github.com/<your-username>/askmynotes, (5) `git init` your existing AskMyNotes folder, make an initial commit, add the remote, and push. Screenshot the GitHub page showing your commit - this is your baseline for the rest of the week.

AI DEEP DIVE PROMPT

Ask an AI: '*Explain, using an analogy from everyday life that a first-year student would recognize, the difference between Git and GitHub. Also list three things Git does that GitHub does not, and three things GitHub adds on top of Git.*'

SUMMARY CARD

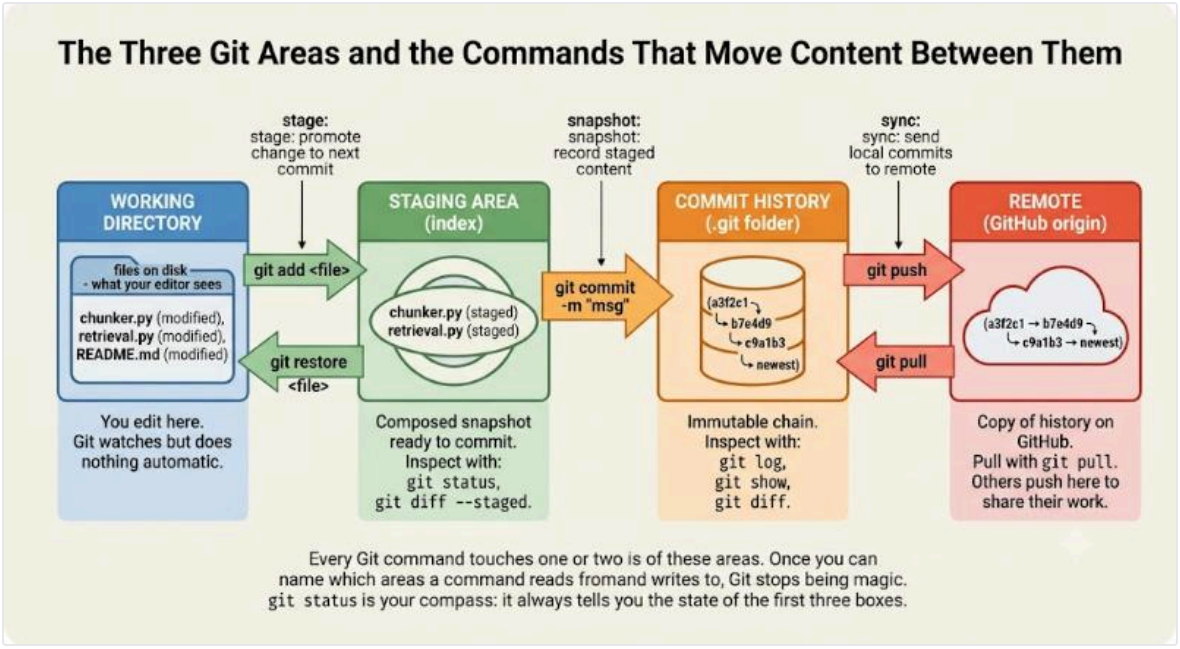
Git is a local, distributed version-control tool that lives in the `.git` folder of every repository. **GitHub** is a hosted platform for sharing Git repositories, plus collaboration features like Pull Requests, issues, and Actions.

Git works offline; GitHub is a web service. Use Git locally, push to GitHub to share.

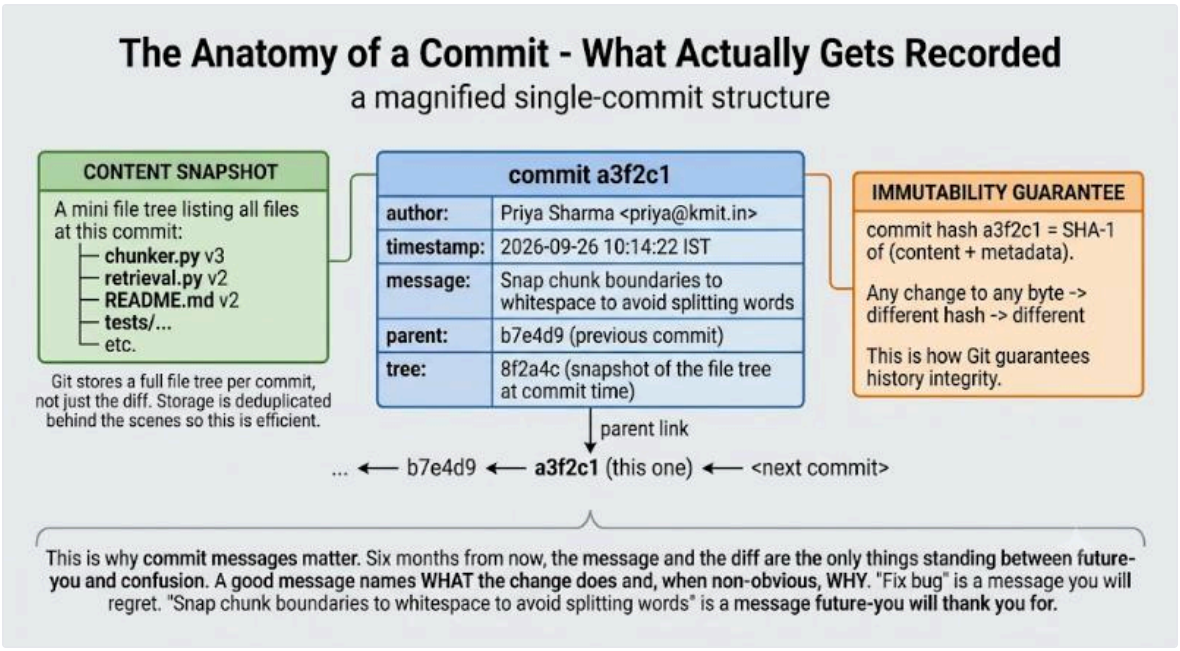
Alternatives to GitHub include GitLab and Bitbucket - your Git skills transfer directly. Set up SSH auth and 2FA on GitHub; use your professional-looking username on your resume; use `git commit` / `git push`, not the web-UI upload button.

THE GIT WORKFLOW - WORKING DIRECTORY, STAGING, COMMIT HISTORY

Claude • AI Assistant



The Three Git Areas and the Commands That Move Content Between Them



The Anatomy of a Commit - What Actually Gets Recorded

bash — The daily Git workflow - edit, inspect, stage, commit, push

```
# ---- START OF SESSION ----
cd ~/projects/askmynotes
git pull                          # bring any teammate's commits down first
git status                        # confirm clean working tree
```

```
# ---- WORK ----
# Open editor, make changes
code chunker.py tests/test_chunker.py
# ... edit ...

# ---- REVIEW ----
git status                      # see WHAT changed
git diff                        # see the actual code changes

# ---- STAGE ----
git add chunker.py tests/test_chunker.py
git status                      # confirm what's staged
git diff --staged               # final review of the composed snapshot

# ---- COMMIT ----
git commit -m "Snap chunk boundaries to whitespace; add tests"
git log --oneline -5            # see your new commit at the top

# ---- SYNC ----
git push                        # send local commits up to GitHub

# ---- OOPS SCENARIOS ----
# Discard a working-directory change you don't want:
git restore path/to/file

# Unstage a file (still keeps your edits, just unstages them):
git restore --staged path/to/file

# Fix the LAST commit's message (must be unpushed, or you rewrite public
history):
git commit --amend -m "Better message"

# Add a file you forgot to include in the last commit (same rule):
git add forgotten_file.py
git commit --amend --no-edit

# Safely undo an already-PUSHED commit by creating a new inverse commit:
git revert a3f2c1
git push
```

Student

I know I need to type `git add` and `git commit` and `git push`, but I don't understand *why* there are three commands. Why not just one 'save to GitHub' button?

Claude • AI Assistant

Great question - this is where most tutorials fail beginners. Git separates the workflow into **three distinct areas** so you can think about your changes in stages rather than flinging every keystroke into history.

Once you see the three areas clearly, every Git command tells you which area it touches, and the whole system becomes obvious.

THE THREE AREAS

Student

Show me the three areas.

Claude • AI Assistant

CONCEPT CARD

From closest-to-you to permanent-record:

1. Working directory. The actual files on your disk that your editor opens. When you type in VS Code, you're editing the working directory. Git watches this folder for changes but does nothing automatic - the files are yours until you tell Git to notice.

2. Staging area (also called the 'index'). A holding pen where you assemble the **next commit**.

This is Git's cleverest idea. When you're done making changes, you decide which changes go into the next snapshot by 'staging' them.

You can stage some files and not others, or stage some *hunks* within a file and not others. The staging area lets you compose a clean, focused commit even after you've made messy edits.

3. Commit history (`.git` folder). The permanent record - a chain of commits, each capturing a snapshot of the staged content plus a message, author, timestamp, and a link to the previous commit.

This chain is what `git log` shows. Once something is committed, it's very hard to lose (see 2.5 Git internals).

A fourth area exists once you push: the **remote** on GitHub. But the interesting design is the local three-area separation.

THE CORE LOOP

Student

Walk me through a typical edit-to-push cycle.

Claude • AI Assistant

EXAMPLE CARD

Ten commands you'll type every day. Adding a bug fix to the AskMyNotes chunker:

bash — Code Example

```bash
# 1. Check current state - what's changed, what's staged, what branch
git status

# 2. Edit files in your editor (chunker.py, tests/test_chunker.py)
# ... changes made ...

# 3. See what changed compared to the last commit
git diff

# 4. Stage the specific files you want in the next commit
git add chunker.py tests/test_chunker.py

# 5. Verify what's staged (vs still just in working directory)
git status
git diff --staged     # show exactly what will be committed

# 6. Commit with a meaningful message
git commit -m "Fix off-by-one in chunk overlap calculation"

# 7. See your new commit at the top of history
git log --oneline -5

# 8. Push to GitHub
git push
```

That's it - the loop. Edit, `git add`, `git commit`, `git push`. Everything else in Git is either a variation or a way to inspect what happened.

WHY STAGING IS WORTH THE EXTRA STEP

Student

The staging area seems annoying. Why can't Git just commit whatever I changed?

Claude • AI Assistant

CONCEPT CARD

Because you don't always want that. Real scenarios where staging saves you:

Scenario 1: mixed changes. You were fixing a bug in `chunker.py` when you noticed a typo in `README.md`. You want two clean commits - one 'Fix chunk overlap' and one 'Fix typo in README' - not one messy 'Various changes'.

Staging lets you `git add chunker.py`, commit, then `git add README.md`, commit again.

Scenario 2: partial file. You edited `retrieval.py` and made two unrelated changes in the same file. `git add -p retrieval.py` (patch mode) lets you interactively stage one **hunk** and skip the other.

You commit the ready part; the rest stays in your working directory for a separate commit later.

Scenario 3: review before commit. `git diff --staged` shows exactly what you're about to snapshot. Catching a debug `print()` you forgot to remove before it enters permanent history is a small superpower - it saves you from having to amend or revert later.

Staging is the difference between a clean, readable Git history and a jumbled one. Every good engineer uses it deliberately.

GIT STATUS - YOUR COMPASS

Student

What's the one command I should run constantly?

Claude • AI Assistant

CONCEPT CARD

`git status`. Run it before and after every operation. It tells you:

- Which branch you're on.

- Which files have changes not yet staged (`Changes not staged for commit`).
- Which files are staged and ready to commit (`Changes to be committed`).
- Which files are untracked - new files Git doesn't know about yet (`Untracked files`).
- Whether your local branch is ahead of or behind the remote (`Your branch is ahead of origin/main by 2 commits`).

Beginners avoid `git status` because it's verbose. Experts run it every 20 seconds because it prevents mistakes. Make it a reflex.

GIT LOG AND GIT DIFF

Student

How do I look at history?

Claude • AI Assistant

EXAMPLE CARD

Two commands, endless variations:

bash — Code Example

```bash
# Full history, one commit per screen
git log

# Compact one-line-per-commit view (extremely useful)
git log --oneline

# Show only the last 10 commits
git log --oneline -10

# Show history as a graph (nice for seeing branches)
git log --oneline --graph --all -20

# See the actual code changes in each commit
git log -p

# Filter by author
git log --author="Priya"

# What changed in a specific file over time
git log --follow -- chunker.py
```

```
# Show the details of one specific commit
git show a3f2c1
```

And `git diff` :

bash — Code Example

```
git diff                  # working directory vs last commit
git diff --staged         # staged changes vs last commit
git diff HEAD~1 HEAD      # last commit vs the one before it
git diff main feature/x   # differences between two branches
```

UNDOING CHANGES

Student

What if I mess up before committing?

Claude • AI Assistant

EXAMPLE CARD

Three levels of undo, matching the three areas:

bash — Code Example

```
# Discard changes in working directory (revert file to last commit's state)
git restore chunker.py
#   Older syntax: git checkout -- chunker.py

# Unstage a file (move it from staging area back to working directory)
git restore --staged chunker.py
#   Older syntax: git reset HEAD chunker.py

# Amend the LAST commit (change its message or add forgotten files)
git commit --amend -m "Better commit message"

# 'Undo' a pushed commit safely by creating a new commit that reverses it
git revert a3f2c1
```

Do not use `git reset --hard` until you understand branches - it's a foot-gun that can wipe work. We cover it carefully in Chapter 2.

APPLIED TO ASKMYNOTES

Student

Show me a real AskMyNotes cycle.

Claude • AI Assistant

EXAMPLE CARD

You just noticed AskMyNotes' PDF chunker sometimes splits mid-word. Fix it and commit:

bash — Code Example

```
cd ~/projects/askmynotes
git status
# On branch main
# nothing to commit, working tree clean

# Edit chunker.py to snap to word boundaries
code chunker.py
# ... editing ...

git status
# Changes not staged for commit:
#   modified:   chunker.py

git diff chunker.py
# ... shows the exact lines changed ...

git add chunker.py
git status
# Changes to be committed:
#   modified:   chunker.py

git diff --staged        # last look before committing
git commit -m "Snap chunk boundaries to whitespace to avoid splitting words"
git log --oneline -3
git push
```

That's a professional-quality commit: focused change, clear message, staged deliberately, pushed to GitHub. Do this a dozen times a day and your repository tells a story anyone can read.

ANALOGY

Think of the three areas as a **kitchen prep line**. Your working directory is the countertop where you chop and mix (messy, in-progress).

The staging area is a plate where you arrange the finished dish (compose, verify). The commit history is the food-service window where finished plates travel out and are recorded on the ticket (permanent).

You wouldn't shove a countertop full of raw ingredients through the window - you plate it first. Same discipline in Git.

REAL WORLD APPLICATION

A tech-lead at a payments company reviews commits during code review. She rejects any commit whose message is 'stuff', 'wip', 'update', or 'fix'.

Rationale: six months from now, when a bug is traced to that commit, the message is the only summary future-you gets. She estimates disciplined commit hygiene saves her team roughly 20 hours a month in archaeology.

Staging area + intentional commits = maintainable history.

WARNING

`git commit -a -m 'msg'` skips the staging area and commits every tracked file that changed. Convenient, but you lose the review step - and it never picks up new files (Git only stages changes to files it already tracks).

Beginners over-rely on `-a`; use it only for tiny, unmistakable one-file changes.

COMMON PITFALL / MISTAKE

Committing large binary or generated files (`node_modules/` , `.pyc` , `.venv/` , model weights, PDFs of test data) into the repo. Repository swells to gigabytes; clones take forever; GitHub complains about file-size limits.

Section 1.5 fixes this with `.gitignore`; for now, be conscious of what you `git add` .

EXAM TIP CARD

Expect a question naming the three areas: 'What are the three main areas Git tracks, and which command moves content between each pair?' Perfect answer: 'Working directory (your edits) -> `git add` -> staging area (composed snapshot) -> `git commit` -> commit history (permanent, in .git). Adding `git push` sends the local history to a remote like GitHub.'

PRACTICE QUIZ

You edited three files: `chunker.py`, `retrieval.py`, and `README.md`. You want to commit only the two Python-file changes together, and commit the README separately. Which sequence?

A) git add . -> git commit -m "changes" -> done

B) git add chunker.py retrieval.py -> git commit -m "..."; git add README.md -> git commit -m "..."

C) git commit chunker.py retrieval.py -> git push

D) Edit README.md later so it doesn't get committed

▼ Reveal Correct Answer

Answer: B. Stage only the files you want in each commit, commit, then repeat for the next batch. This is exactly the discipline the staging area exists for.

EXERCISE

In your AskMyNotes repo, make three small edits in three different files (change a comment in one, add a print statement in another, add a line to README). Then: (a) stage and commit ONE file with a good message, (b) stage and commit the SECOND file with its own message, (c) discard the third change using `git restore`.

Confirm with `git log --oneline -5` and `git status` at each step.

AI DEEP DIVE PROMPT

Ask an AI: '*Explain the Git working directory, staging area, and commit history to me using a real-world analogy. Then walk through a commit workflow for a Python project where I want to commit a bug fix separately from an unrelated documentation update.*'

SUMMARY CARD

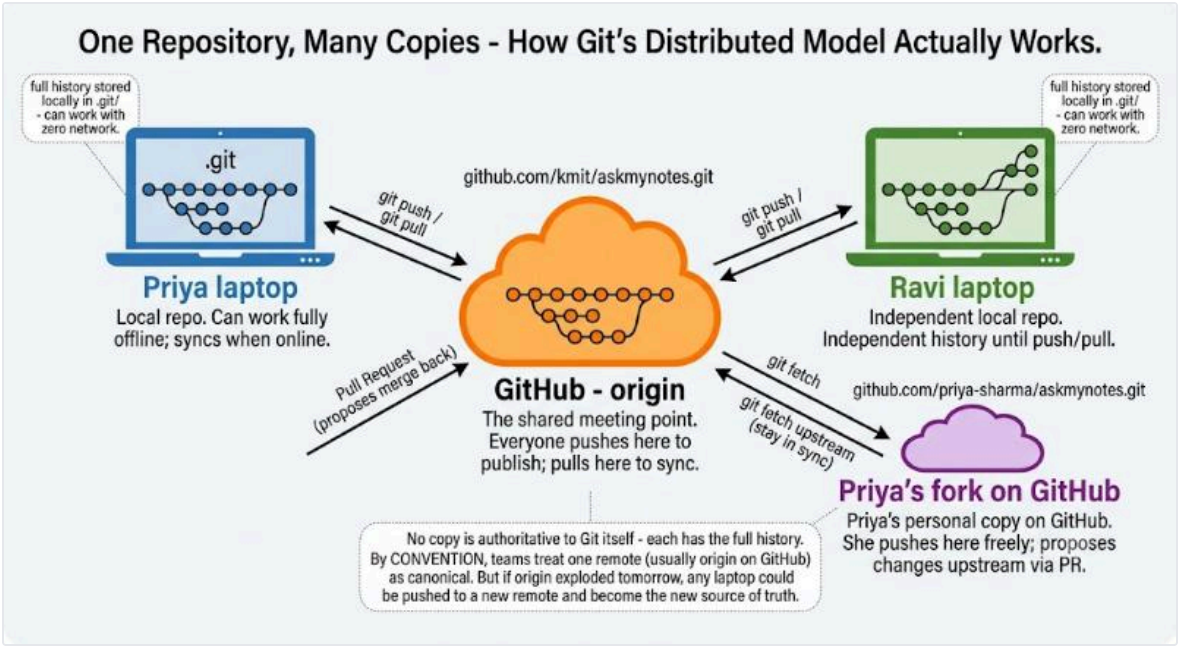
Git tracks three local areas: **working directory** (files you edit), **staging area** (composed snapshot for the next commit), and **commit history** (permanent record in `.git`). The core loop: edit -> `git add` -> `git commit` -> `git push`.

Use `git status` compulsively; `git diff` and `git diff --staged` before every commit; `git log --oneline --graph` to inspect history. Undo levels: `git restore` (discard), `git restore --staged` (unstage), `git commit --amend` (fix last commit), `git revert` (undo a pushed commit safely).

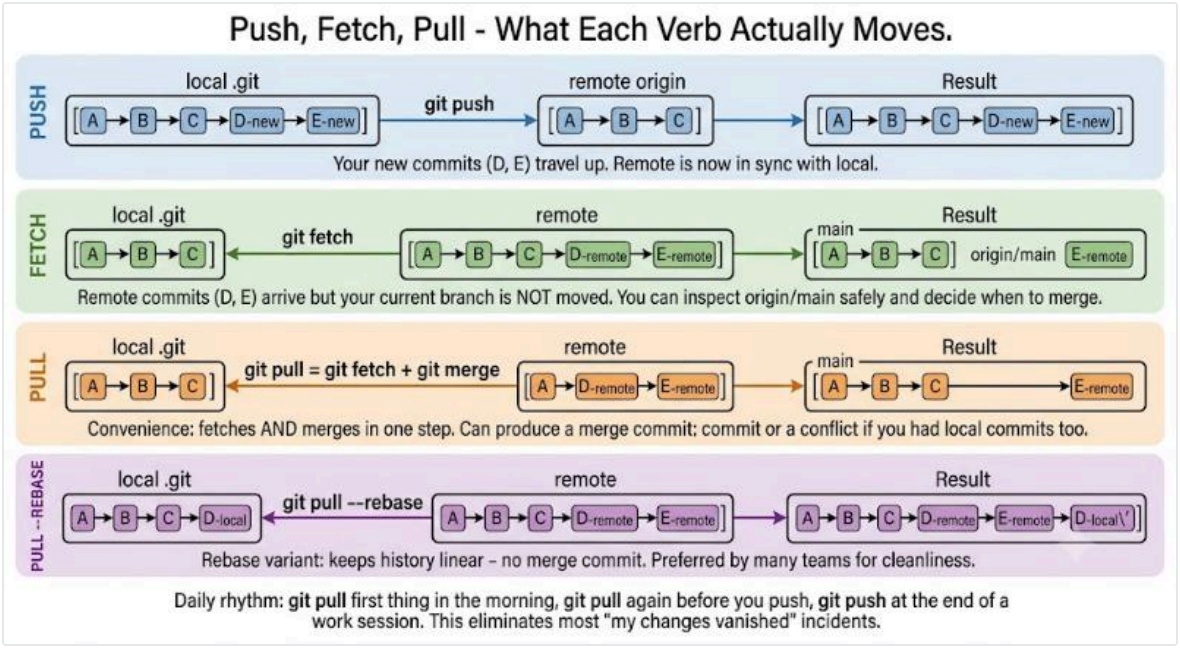
The staging area is what lets you build a clean, story-like history.

REPOSITORIES AND REMOTES - CLONING, PUSHING, PULLING

Claude • AI Assistant



One Repository, Many Copies - How Git's Distributed Model Actually Works



Push, Fetch, Pull - What Each Verb Actually Moves

bash — Remotes, cloning, and multi-remote setups - the operational vocabulary

```
# ---- CLONE an existing GitHub repo ----
cd ~/projects
git clone git@github.com:priya-sharma/askmynotes.git
cd askmynotes
```

```
# Inspect: full history and all remote branches are already local
git remote -v                               # origin is set automatically
git branch -a                               # local + remote branches
git log --oneline -5

# ---- WORK ON A BRANCH ----
git checkout -b feature/rerank              # create + switch to a branch
# ... edit, add, commit ...
git push -u origin feature/rerank           # first push sets upstream
tracking
# Now future pushes on this branch are simply: git push

# ---- STAY IN SYNC (daily rhythm) ----
git checkout main
git pull --rebase                           # bring remote commits down
cleanly
# Do work on feature/rerank ...
git push

# ---- INSPECT a remote's state WITHOUT merging ----
git fetch origin
git log --oneline origin/main -10           # see what's on the server
git diff main..origin/main                  # what would arrive if I pulled?

# ---- ADD A SECOND REMOTE (mirror to internal GitLab) ----
git remote add gitlab git@gitlab.mango.internal:analytics/askmynotes.git
git push gitlab main
# List all remotes:
git remote -v
# origin  git@github.com:...       (fetch)
# origin  git@github.com:...       (push)
# gitlab  git@gitlab.mango...      (fetch)
# gitlab  git@gitlab.mango...      (push)

# ---- FORK CONTRIBUTION FLOW (open source) ----
# 1. Fork on github.com in the browser
# 2. Clone your fork:
git clone git@github.com:priya-sharma/famous-oss-project.git
cd famous-oss-project
# 3. Track the ORIGINAL as 'upstream' so you can stay in sync:
git remote add upstream git@github.com:original-owner/famous-oss-project.git
git fetch upstream
git checkout -b fix/typo-in-readme upstream/main
# 4. Edit, commit, push to YOUR fork:
git push -u origin fix/typo-in-readme
```

```
# 5. Open a Pull Request on GitHub: your fork's branch -> original repo's
main

# ---- REMOVING / RENAMING A REMOTE ----
git remote rename gitlab internal
git remote remove internal
```

Student

We covered the local Git workflow. Now let's dig into the remote side - how do repositories move between my laptop, GitHub, and my teammates' machines?

Claude • AI Assistant

A **repository** is any folder containing a `.git` directory - Git treats it as a self-contained project with its own history. A **remote** is a named URL pointing to another copy of that repository, typically hosted on GitHub.

Two big verbs move commits between repositories: **push** sends your local commits up, **pull** brings remote commits down. Most of Git's power over centralized systems comes from the fact that every repository - yours, your teammate's, GitHub's - has the full history and can independently share with any of the others.

STARTING A REPOSITORY - INIT VS CLONE

Student

Two ways to get started?

Claude • AI Assistant

CONCEPT CARD

Two paths, depending on where the code is:

`git init` - start from a local folder. You already have code on your laptop (or an empty folder for a new project); you want to bring Git into it. `cd` into the folder, run `git init`, and Git creates a hidden `.git/` subdirectory.

The folder is now a repository. You typically then create a matching empty repo on GitHub and wire the two together with `git remote add`.

`git clone <url>` - copy an existing remote repository. Someone else's repo (or your own from another machine) already exists on GitHub. `git clone git@github.com:priya-sharma/askmynotes.git` downloads a full copy including all history, checks out the default branch, and automatically registers the source URL as the remote named `origin`.

This is what you use to pick up a project from GitHub.

Rule of thumb: **starting fresh on your laptop -> `git init` then wire remote. Joining an existing GitHub repo -> `git clone`**.

THE CONCEPT OF A REMOTE

Student

A remote is just a URL?

Claude • AI Assistant

CONCEPT CARD

Technically yes - a **remote** is a named alias for a URL where a copy of your repository lives. `git remote -v` lists them:

bash — Code Example

```
git remote -v
# origin  git@github.com:priya-sharma/askmynotes.git (fetch)
# origin  git@github.com:priya-sharma/askmynotes.git (push)
```

By convention, the default remote is called **`origin`**. But you can have multiple remotes with different names - useful when you want to sync a project between GitHub and, say, a company-internal GitLab, or when working with **forks** (see below):

bash — Code Example

```
git remote add gitlab git@gitlab.mango.internal:analytics/askmynotes.git
git remote add upstream git@github.com:kmit/askmynotes.git
git push origin main       # push to GitHub
git push gitlab main       # also push to internal GitLab
```

Remotes are metadata - they cost nothing to configure; nothing is transferred until you `push` or `fetch`.

PUSH, FETCH, PULL

Student

What's the difference between fetch and pull?

Claude • AI Assistant

CONCEPT CARD

Three verbs to know precisely:

`git push` - upload your local commits to the remote. Standard: `git push` (assumes current branch's tracked remote).

Explicit: `git push origin main`. First-time push of a new branch: `git push -u origin feature/rerank` (the `-u` sets 'upstream' so future pushes/pulls don't need the branch name).

`git fetch` - download remote commits into your local repository but **do not** merge them into your working branch. After `git fetch origin`, you can inspect remote changes with `git log origin/main` before deciding to integrate them.

Fetch is safe - it never touches your working directory.

`git pull` - shorthand for `git fetch` immediately followed by `git merge origin/<branch>`. Convenient but can cause an unexpected merge commit or conflict if you had local commits.

Many teams prefer `git pull --rebase` (replay your local commits on top of the freshly pulled remote commits) for a cleaner history.

Daily workflow rule: pull before you start work, pull again before you push. This avoids most 'someone else pushed while I was working' surprises.

CLONING YOUR FIRST REPO

Student

Show me a clone in action.

Claude • AI Assistant

EXAMPLE CARD

Cloning AskMyNotes from GitHub to a fresh machine:

bash — Code Example

```bash
# Move to where you keep projects
cd ~/projects

# Clone - creates a subfolder called askmynotes/ with full history
git clone git@github.com:priya-sharma/askmynotes.git

cd askmynotes
git remote -v                    # origin is set automatically
git log --oneline -5             # full history is already local
git branch -a                    # -a shows local AND remote branches
#   * main
#     remotes/origin/HEAD -> origin/main
#     remotes/origin/main
#     remotes/origin/feature/rerank
```

Notice: cloning gives you the full history and all remote branches for free. You can immediately `git checkout feature/rerank` to switch to any branch your teammates were working on.

FORKS VS CLONES

Student

I keep hearing 'fork'. What's that?

Claude • AI Assistant

CONCEPT CARD

A **fork** is a GitHub feature (not a Git feature): you create your own copy of someone else's public repo on GitHub. You can push to your fork freely without affecting the original; you can open a **Pull Request** back to the original repo to propose your changes.

The typical open-source contribution flow:

1. Fork the repo on GitHub -> now `github.com/priya-sharma/famous-open-source-project` exists.

2. Clone your fork: `git clone git@github.com:priya-sharma/famous-open-source-project.git` .

3. Add the original as `upstream`: `git remote add upstream git@github.com:original-owner/famous-open-source-project.git` . This lets you `git pull upstream main` to stay in sync with the original.

4. Branch, edit, commit, push to YOUR fork: `git push origin feature/my-fix` .

5. Open a Pull Request from your fork's branch back to the original repo's main branch. Maintainers review; if accepted, they merge it.

Every open-source contribution on GitHub uses this pattern.

THE TRACKING BRANCH RELATIONSHIP

Student

What is 'upstream' and 'tracking'?

Claude • AI Assistant

CONCEPT CARD

A local branch can be **tracking** a remote branch. When it is, `git status` shows things like 'Your branch is ahead of origin/main by 2 commits' and `git push` / `git pull` don't need arguments. The tracking relationship is set:

- Automatically when you `git clone` (main tracks origin/main).
- Automatically when you `git checkout -b feature/x origin/feature/x` .
- Explicitly with `git push -u origin feature/x` (first push of a new branch).
- Retroactively with `git branch --set-upstream-to=origin/main` .

You'll know a branch isn't tracking anything when `git push` complains: `fatal: The current branch has no upstream branch. To push the current branch and set the remote as upstream, use: git push --set-upstream origin <branch>` . Do what it says.

ANALOGY

Think of your local repository as your **home library** and the remote as the **university library**. `git clone` = check out a copy of every book, walk home with them.

`git pull` = pop over and pick up the new arrivals. `git push` = donate your new books back to the university.

Multiple remotes = you also visit the city library sometimes; both know about your work. A fork is like photocopying the university's books to keep in your own home library and only 'donating' back specific chapters you improved (via Pull Request).

REAL WORLD APPLICATION

A team of six data engineers works on the same Snowflake pipeline repo. Each morning, their first Git operation is `git pull --rebase`; their last operation before signing off is `git push`.

Between those two, they branch, commit, and open PRs. This simple discipline - pull first, push last - eliminates about 95% of the 'someone else's changes wiped mine' incidents that plague less-disciplined teams.

⚠ WARNING

`git push --force` (or `-f`) rewrites remote history. On a branch other people use, this is destructive: their work can vanish.

Use `git push --force-with-lease` instead when you must force-push - it fails safely if someone else pushed since your last fetch. Never force-push to `main` on a shared repo without explicit team coordination.

COMMON PITFALL / MISTAKE

Beginners create the GitHub repo *with* a README/gitignore/license *and* also have local files, then hit push conflicts. Two rules to avoid this: (1) if you have local files, create the GitHub repo EMPTY and push to it; (2) if you want GitHub's boilerplate, clone the empty-with-README repo first and add your files inside the clone.

EXAM TIP CARD

Interviewers may ask 'walk me through cloning a repo, making a change, and getting it merged.' Ideal answer: ' `git clone <url>` to get a local copy; `git checkout -b feature/x` to branch; edit; `git add` / `git commit`; `git push -u origin feature/x`; open a Pull Request on GitHub; get review; merge.' Naming `-u origin feature/x` on first push signals fluency.

PRACTICE QUIZ

You just cloned a repo and made three commits on the main branch locally. You run `git push`. What happens if the remote main has moved (someone else pushed) while you were working?

A) Your commits overwrite the remote ones

B) Git rejects the push and asks you to pull first

C) Git silently merges them

D) Git deletes your local commits

▼ Reveal Correct Answer

Answer: B. Git protects the remote from being clobbered - it rejects a non-fast-forward push. You must `git pull` (which may merge or ask you to resolve conflicts), then push again.

EXERCISE

(a) `git clone` your AskMyNotes repo into a NEW folder (e.g., `~/tmp/askmynotes-copy`). (b) In that copy, make a small change, commit, push.

(c) Go back to your ORIGINAL folder, run `git status` - it doesn't know about the change yet. Run `git pull` and watch the new commit arrive.

This is exactly how a teammate's changes reach you.

AI DEEP DIVE PROMPT

Ask an AI: '*Explain the difference between git init and git clone, and walk through the fork-clone-branch-PR workflow used to contribute to open-source projects on GitHub. Give commands for each step.*'

SUMMARY CARD

A **repository** is a folder with a `.git` subdirectory. A **remote** is a named URL pointing to another copy of it (`origin` by default).

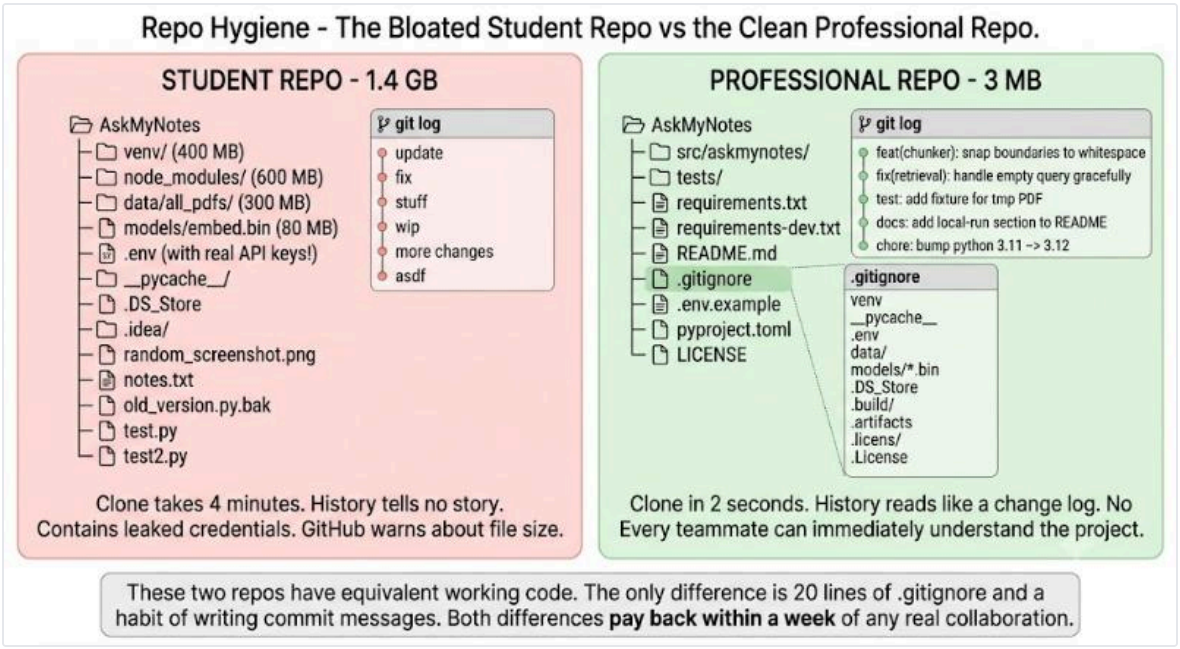
Start with **`git init`** (new local folder) or **`git clone <url>`** (existing remote). Push local commits with **`git push`**; download remote commits with **`git fetch`** (safe, no merge) or **`git pull`** (fetch + merge).

Set tracking with **`git push -u origin <branch>`** on first push of a new branch. Forks (a GitHub feature) enable open-source contribution via the fork -> clone -> branch -> push -> PR flow.

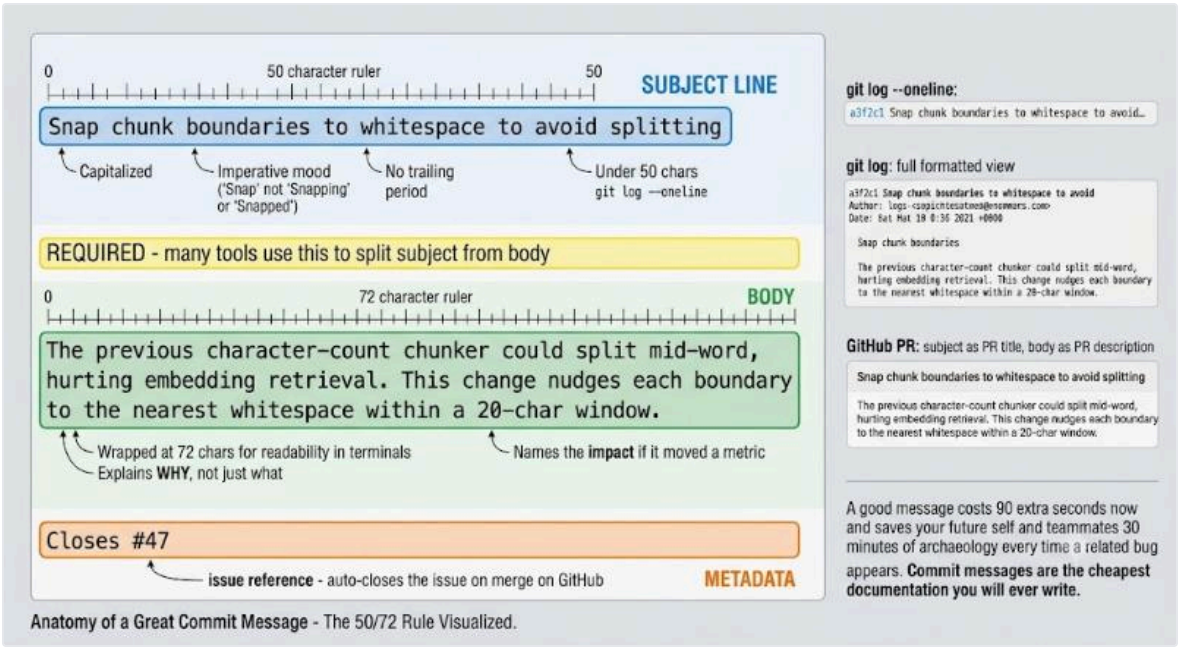
Daily discipline: pull before you start, pull before you push.

.GITIGNORE AND COMMIT HYGIENE - WHAT NEVER TO COMMIT AND HOW TO WRITE MESSAGES

Claude • AI Assistant



Repo Hygiene - The Bloated Student Repo vs the Clean Professional Repo



Anatomy of a Great Commit Message - The 50/72 Rule Visualized

gitignore — A ready-to-use .gitignore for a Python + ML project like AskMyNotes

```
# ============================================================
# .gitignore for a Python + ML project (adapt as needed)
# Start from github.com/github/gitignore/blob/main/Python.gitignore
# then add project-specific entries below.
# ============================================================

# --- Byte-compiled / optimized ---
__pycache__/
```

```
*.py[cod]
*$py.class

# --- Distribution / packaging ---
build/
dist/
*.egg-info/
.eggs/

# --- Virtual environments ---
.venv/
venv/
env/
ENV/

# --- Testing / coverage / linting ---
.pytest_cache/
.coverage
.coverage.*
htmlcov/
.ruff_cache/
.mypy_cache/
.tox/

# --- Jupyter / IPython ---
.ipynb_checkpoints/
*.ipynb_checkpoints

# --- Secrets and env files ---
.env
.env.*
!.env.example              # keep the template, ignore the real ones
*.pem
*.key
secrets.yml
credentials.json

# --- OS / editor ---
.DS_Store
Thumbs.db
.idea/
.vscode/
!.vscode/settings.shared.json    # negate if you share workspace settings

# --- Logs ---
*.log
```

```
logs/

# --- ML artifacts / large data (use DVC or Git LFS if needed) ---
data/
!data/README.md            # keep the readme explaining what data goes here
models/
*.bin
*.onnx
*.h5
*.pkl
*.parquet
*.sqlite
*.db

# --- MLflow (see Chapter 4) ---
mlruns/
mlartifacts/
```

bash — Commit-message discipline and the 'oh no I committed a secret' recovery

```
# --- Writing a good commit message with an editor (multi-line) ---
git commit                          # opens your $EDITOR
# Type the subject on line 1 (<=50 chars), blank line, body wrapped at 72

# --- Or all in one line for simple commits ---
git commit -m "Fix chunker splitting mid-word"

# --- Conventional Commits format (optional but popular) ---
git commit -m "feat(chunker): snap boundaries to whitespace"
git commit -m "fix(retrieval): handle empty query gracefully"
git commit -m "docs: add local-run section to README"
git commit -m "test: add pytest fixture for tempfile PDF"
git commit -m "chore: bump python 3.11 -> 3.12"

# --- Amend the last commit's message BEFORE pushing ---
git commit --amend -m "Better message"

# ============================================================
# EMERGENCY: You committed a secret to a public repo. GO!
# ============================================================
# STEP 1: ROTATE the credential on the service first!
#         AWS: create new access key, delete old one
#         OpenAI: revoke and issue a new API key
#         Snowflake: rotate the user's key-pair
#         GitHub token: revoke in Settings -> Developer settings
```

```
#         (Do this BEFORE touching the repo - the old value is dead.)

# STEP 2: Stop tracking the file (keeps your local copy)
git rm --cached .env
echo ".env" >> .gitignore
git add .gitignore
git commit -m "Stop tracking .env; add to gitignore"
git push

# STEP 3: Purge from ALL history (rewrites history - coordinate first!)
# Install: pip install git-filter-repo
git filter-repo --path .env --invert-paths --force
git push --force-with-lease origin main

# STEP 4: Add prevention - pre-commit hook (details in Chapter 4)
#   Tools: pre-commit + gitleaks or detect-secrets
# See https://pre-commit.com and https://github.com/gitleaks/gitleaks
```

Student

My commit messages are usually 'update' or 'fix stuff'. And I've definitely committed a `.env` file at least once. Time to fix these habits?

Claude • AI Assistant

Long past time. Two disciplines separate a professional codebase from a student one: knowing **what should never be committed** (managed by `.gitignore`) and knowing **how to write commit messages** future readers can use.

This subsection nails both. By the end you'll have a solid `.gitignore` for AskMyNotes and a mental template for messages that read like documentation.

WHAT .GITIGNORE DOES

Student

Show me .gitignore.

Claude • AI Assistant

CONCEPT CARD

`.gitignore` is a plain text file at the root of your repo listing patterns that Git should **ignore** - meaning it won't track them, `git add .` won't stage them, and `git status` won't nag you about them. It's the tool for keeping build artifacts, secrets, and noise out of your history.

A minimal `.gitignore` for a Python project:

gitignore — Code Example

```
# --- Python ---
__pycache__/
*.py[cod]
*.egg-info/
.pytest_cache/
.ruff_cache/
.coverage
htmlcov/

# --- Virtual environments ---
.venv/
venv/
env/

# --- Secrets and env files ---
.env
.env.*
!.env.example       # ! negates - keep the example template
*.pem
secrets.yml

# --- OS / editor ---
.DS_Store
Thumbs.db
.idea/
.vscode/settings.json

# --- Data and models (use DVC or Git LFS if needed) ---
data/
models/*.bin
*.parquet
*.sqlite

# --- Logs ---
*.log
logs/
```

Rules: `#` starts a comment; `*.pyc` matches any .pyc file at any depth; `/` at the end matches directories; `!pattern` negates (include something that a broader pattern excluded).

THE GITHUB/GITIGNORE TEMPLATES

Student

Do I have to write this from scratch every time?

Claude • AI Assistant

No - GitHub hosts a curated set of language-specific `.gitignore` templates at **`github.com/github/gitignore`**. When you create a new GitHub repo, one of the setup checkboxes asks 'Add .gitignore' with a language dropdown. Or grab the raw Python template:

bash — Code Example

```
curl -o .gitignore
https://raw.githubusercontent.com/github/gitignore/main/Python.gitignore
```

Start from that template, then add project-specific entries (data folders, model files, personal notes). This saves you from missing important entries the community has already catalogued.

THE 'OH NO, I COMMITTED A SECRET' RECOVERY

Student

What if I already committed something I shouldn't have?

Claude • AI Assistant

⚠ **WARNING**

Serious moment. **Adding the file to `.gitignore` does NOT remove it from history** - it only prevents future commits. The file is still there in every commit that included it, and anyone with the repo can find it. To handle a leak:

1. If it's a secret (API key, password, token) - ROTATE IT FIRST. Assume it's compromised. Delete/regenerate the key on the service side (AWS, OpenAI, Snowflake, whatever) before you do anything to the repo. The old value must be dead.

2. Remove the file from the current commit:

bash — Code Example

```
git rm --cached .env          # stop tracking; keep local copy
echo ".env" >> .gitignore
git add .gitignore
git commit -m "Stop tracking .env; add to gitignore"
git push
```

3. If the secret was pushed publicly, also purge it from ALL history with a tool like `git filter-repo` or the BFG Repo-Cleaner. This rewrites history - coordinate with any collaborators before force-pushing.

Prevention is easier than cure: set up `.gitignore` on day one; use pre-commit hooks (see Chapter 4) to catch secrets before they're committed.

COMMIT-MESSAGE HYGIENE

Student

Beyond avoiding 'fix stuff', what makes a message actually good?

Claude • AI Assistant

CONCEPT CARD

The community-standard template - popularized by a widely-cited essay by Chris Beams - is the **50/72 rule** plus a few conventions:

Subject line (first line, appears in `git log --oneline`):

- Max 50 characters.
- Capitalized, no trailing period.
- Imperative mood - **'Fix chunker bug'**, not 'Fixed' or 'Fixes' or 'Fixing'. Think of it as completing 'If applied, this commit will _____'.

Body (optional, but recommended for anything non-obvious):

- Blank line between subject and body (matters - many tools use this separator).
- Wrap at ~72 characters per line.

- Explain **why** the change was made and any important context. The code shows what changed; the message explains why.
- Reference issues if applicable: 'Closes #42'.

Example of a good commit:

code — Code Example

```
Snap chunk boundaries to whitespace to avoid splitting words

The previous character-count chunker could split mid-word, which
hurt embedding retrieval quality (partial words don't have good
embeddings). This change nudges each boundary to the nearest
whitespace within a 20-character window. If no whitespace is
found, we fall back to the original character-count boundary.

Retrieval MRR on the internal FAQ eval improved from 0.71 to 0.78.

Closes #47
```

Read that in six months when a related bug pops up - you have the *why*, the metric that motivated it, and the fallback logic all documented. That's the value of a real commit message.

CONVENTIONAL COMMITS - A POPULAR OPTIONAL FORMAT

Student

I've seen commits like 'feat: add re-ranker' or 'fix(chunker): snap to whitespace'. What's that?

Claude • AI Assistant

CONCEPT CARD

Conventional Commits is an optional convention where the subject starts with a *type* and optional *scope*:

code — Code Example

```
feat(chunker): snap boundaries to whitespace
fix(retrieval): handle empty query gracefully
docs: add README section on running locally
chore: bump python from 3.11 to 3.12
```

```
refactor: extract embedding call into its own function
test: add pytest fixture for tmp PDF
```

Types: `feat` (new feature), `fix` (bug fix), `docs`, `chore` (build/tooling), `refactor` (no behavior change), `test`, `perf`, `ci`. Scope in parens is optional and identifies the affected area.

Why teams adopt it: enables automated changelog generation, semantic-version bumps, and easy filtering (`git log --grep="^fix"`). Not required, but if your team uses it, follow it.

ATOMIC COMMITS

Student

One more habit?

Claude • AI Assistant

CONCEPT CARD

An **atomic commit** does ONE thing. If your subject line needs an 'and', it's probably two commits. Bad: 'Fix chunker AND update README AND add tests'. Good: three separate commits. Benefits:

- **Easier review**: reviewers can approve one commit while asking questions about another.
- **Easier revert**: if just one part broke, `git revert` that one commit and leave the others.
- **Easier archaeology**: `git log --follow file.py` returns focused, readable history.

The **staging area** you learned in 1.3 is what makes atomic commits practical - you compose the commit deliberately, one intention at a time.

ANALOGY

`.gitignore` is like the **do-not-pack list** when moving apartments: perishables, houseplants, hazardous chemicals. You know they shouldn't go in the truck; the list makes sure they don't get accidentally boxed up.

Good commit messages are like **labeling every moving box**: 'Kitchen - dishes' beats an unlabeled box every time. On unpacking day (or debugging day, six months from now), you'll thank yourself.

REAL WORLD APPLICATION

A student's public GitHub project was scanned by an AWS-credential bot within four minutes of a commit that included `.env` with an AWS access key. The bot spun up EC2 instances for cryptocurrency mining.

The student got a $2,300 AWS bill by the next morning. AWS forgave most of it after investigation, but the takeaway is real: **secrets in public repos are exfiltrated within minutes, not days**.

`.gitignore` on day one is not optional.

⚠ WARNING

Do not put personal `.gitignore` entries (like your specific editor's cache files) in the project's `.gitignore` - use `~/.config/git/ignore` (global) for those instead. The project's `.gitignore` should contain patterns that apply to every developer working on the project, not your personal machine idiosyncrasies.

COMMON PITFALL / MISTAKE

Committing `venv/` or `node_modules/`. These regenerate deterministically from `requirements.txt` / `package-lock.json`; committing them adds gigabytes to your repo and creates massive merge noise. **Rule: if it can be rebuilt from a lockfile, it goes in `.gitignore`.**

EXAM TIP CARD

Interview scenario: 'You accidentally committed AWS credentials to a public repo yesterday. What do you do?' Expected answer: (1) rotate the key immediately on AWS - don't touch the repo first; (2) remove the file from tracking with `git rm --cached`; (3) add it to `.gitignore`; (4) purge from full history with `git filter-repo` or BFG; (5) force-push (coordinating with team); (6) add a pre-commit hook to prevent recurrence.

Naming step 1 first is what signals security-mindedness.

PRACTICE QUIZ

Which of the following is the WORST commit message?

A) 'Fix chunker off-by-one so overlap includes trailing token'

B) 'Add pytest fixture for tempfile-based PDF'

C) 'stuff'

D) 'feat(retrieval): add cross-encoder re-ranking on top-K'

▼ Reveal Correct Answer

Answer: C. Uninformative, lowercase, tells future-you nothing. A, B, D are all serviceable to

excellent.

EXERCISE

Add a `.gitignore` to AskMyNotes covering: Python bytecode, virtual envs, `.env` files, `.pytest_cache/`, `.DS_Store`, `data/`, `models/*.bin`. Commit it with a message that follows the 50/72 rule.

Then look at your last 10 commits with `git log --oneline -10` and honestly grade them: which would you be embarrassed to show in a code review?

AI DEEP DIVE PROMPT

Ask an AI: '*Generate a comprehensive .gitignore for a Python project using FastAPI, pytest, and MLflow, with an embeddings/data folder that must not be committed. Then review these three commit messages and rewrite the weak ones: "update", "fix pdf issue", "work in progress*".'

SUMMARY CARD

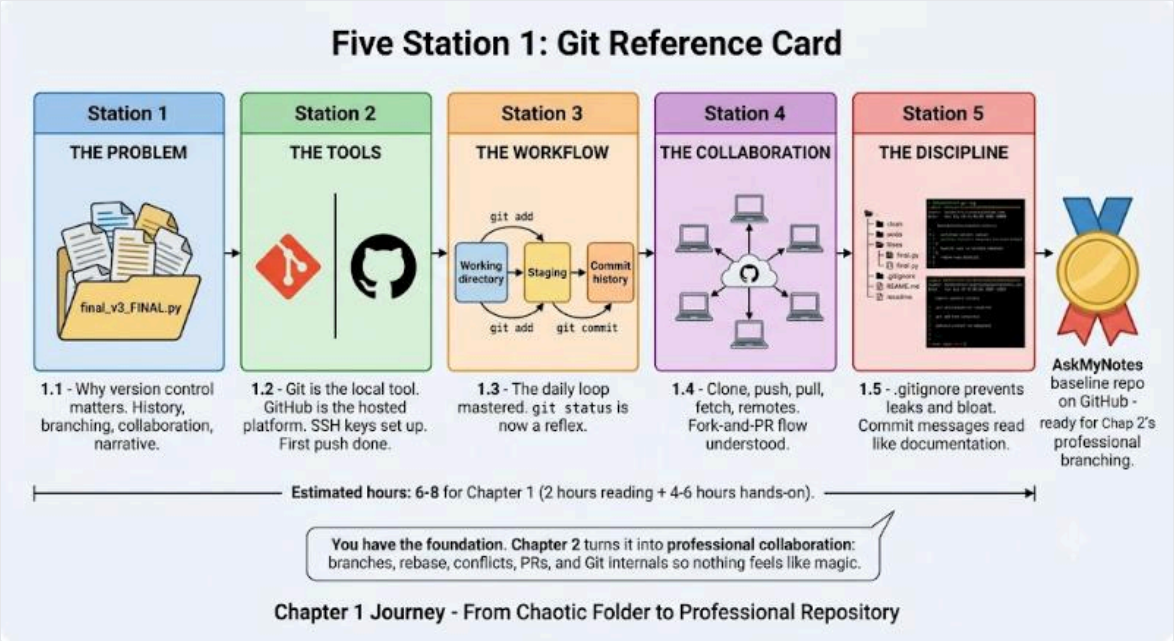
.gitignore lists patterns Git should ignore - keep out: secrets, build artifacts, virtual envs, dependency directories, OS/editor cruft, logs, large data/models. Start from `github/gitignore` templates; add project-specific entries.

If you already committed a secret: rotate the key FIRST, then untrack + purge from history. **Commit messages**: 50-char imperative subject, blank line, 72-char body, explain the WHY.

Consider **Conventional Commits** (`feat:`, `fix:`, `docs:`) for machine-readable history. **Atomic commits** - one intent per commit - make review, revert, and archaeology dramatically easier.

CHAPTER 1 REVIEW & SUMMARY

Claude • AI Assistant



Chapter 1 Journey - From Chaotic Folder to Professional Repository

bash — Chapter 1 command reference - the twelve verbs of Git basics

```
# ---- Setup (once per machine) ----
git --version
git config --global user.name  "Priya Sharma"
git config --global user.email "priya.sharma@kmit.in"
git config --global init.defaultBranch main

# ---- Start a project ----
git init                                        # new local repo
git clone git@github.com:me/askmynotes.git      # copy an existing repo

# ---- The daily loop ----
git status
git diff
git add <file>          # or:  git add .    to stage everything
git diff --staged
git commit -m "Concise 50-char subject"
git log --oneline -10
git push                # or:  git push -u origin <branch>  on first push

# ---- Sync with remote ----
git fetch origin                                  # download without merging
git pull                                          # download and merge
git pull --rebase                                 # download and replay locals

# ---- Undo ----
```

```
git restore <file>                              # discard working-dir edits
git restore --staged <file>                     # unstage
git commit --amend -m "Better message"          # fix last commit
(unpushed!)
git revert <commit>                             # safe undo of a pushed
commit

# ---- Remotes ----
git remote -v
git remote add origin git@github.com:me/askmynotes.git
git remote add upstream git@github.com:kmit/askmynotes.git

# ---- Chapter 2 preview ----
git branch                                      # list branches
git checkout -b feature/rerank                  # create + switch to branch
git merge feature/rerank                        # merge branch into current
git rebase main                                 # replay commits onto main

# The above 25 commands cover ~85% of daily Git use. Chapter 2 refines
# the branching commands; Chapter 3 hooks Git into CI; Chapter 4 wires
# in quality gates and deployment; Chapter 5 adds MLflow for ML lifecycle.
```

CHAPTER SUMMARY

You have covered the Git and GitHub foundation completely. **Section 1.1** made the case for version control - solving the 'final*v3*FINAL.py' chaos with history, branching, collaboration, and narrative.

Section 1.2 clarified the crucial distinction between Git (the local tool that lives in every `.git/` folder) and GitHub (the hosted platform that adds Pull Requests, Actions, and issues on top of Git repos). **Section 1.3** taught the three areas Git tracks - working directory, staging area, commit history - and the daily loop: edit -> `git add` -> `git commit` -> `git push` .

Section 1.4 covered repositories and remotes: how to `init` a new repo or `clone` an existing one, how to push and pull, and the fork-based open-source contribution flow. **Section 1.5** locked in the two habits that separate hobbyist repos from professional ones: `.gitignore` (never commit secrets, build artifacts, or bloat) and commit-message hygiene (50/72 rule, imperative mood, explain the why).

THE ONE-PAGE MENTAL MODEL

Version control is a **distributed history layer** for code. Git manages that history in the `.git/` folder of every repository - fully offline, resilient, cheap to branch.

GitHub is where teams meet: they push copies of their Git repositories there to share, and layer collaboration features (PRs, issues, CI) on top. The daily loop is small and consistent - edit, stage, commit, push - and the disciplines that matter most are keeping garbage out of the repo and writing commit messages that read like documentation.

WHAT'S NEXT

Chapter 2 takes Git from single-developer basics into **professional collaboration**: branching strategies (feature branches, GitFlow, trunk-based), rebasing vs merging, resolving conflicts (the anxiety-inducing skill worth extra practice), Pull Request etiquette, and enough Git internals that nothing feels like magic and nothing is unrecoverable.

SELF-CHECK

Before moving on, verify you can:

1. Explain the difference between Git and GitHub in one sentence each.
2. Name the three local areas Git tracks and the command that moves content between each pair.
3. Run `git init`, make a commit, add a GitHub remote, and push - without looking up any command.
4. Clone a repo, create a branch, commit on it, push it, and open a Pull Request.
5. Write a `.gitignore` for a Python project from scratch.
6. Write a commit message that follows the 50/72 rule and explains WHY.
7. Explain what you would do (in the right order) if you accidentally committed an API key to a public repo.

If any of the seven are shaky, re-read the relevant subsection - Chapter 2 will assume all seven.

CHAPTER 1 EXERCISE - THE ASKMYNOTES BASELINE

Do this before starting Chapter 2:

1. Ensure Git is installed and configured with your GitHub email.
2. Set up SSH auth with GitHub; enable 2FA.
3. `git init` your AskMyNotes folder (or `git clone` if it's already on GitHub).
4. Add a comprehensive `.gitignore` (use the template from 1.5).
5. Make at least 3 commits with 50/72-quality messages.
6. Push to a GitHub repo whose name you would be comfortable putting on your resume.
7. Take a screenshot of the GitHub page showing your commit history - this is your baseline; Chapter 5's capstone will build on it.

SUMMARY CARD

> Chapter 1 covered **Git fundamentals**: what version control is and why it matters; the sharp distinction between Git (local tool) and GitHub (hosted platform); the three-area workflow (working directory -> staging -> commit history); repositories and remotes (init vs clone; push, fetch, pull; forks); and the professional-hygiene disciplines of `.gitignore` and commit messages. AskMyNotes is now a properly-versioned repo on GitHub, ready to become the vehicle for professional branching, testing, CI/CD, and MLOps over the rest of the week.